

Real-Time Quranic Recitation Assessment on Mobile Devices Using Streaming Inference and Model Compression

Abdelrahman Ayman Amin Mohamed Abdullah Haytham Mohamed Salah Hedeya

Faculty of Engineering, Cairo University

Giza, Egypt

abdelrahman.akefady@gmail.com, hedeya@gmail.com

Abstract

Automatic Quranic recitation assessment requires accurate detection of phonetic and Tajweed-related pronunciation errors while providing low-latency feedback. However, most speech recognition models are designed for offline processing and are too large for deployment on consumer mobile devices. We present a real-time Quranic recitation assessment system based on a multi-head Wav2Vec2-BERT model trained to jointly predict phonemes and Tajweed *Sifat* attributes using Connectionist Temporal Classification. To enable streaming inference without retraining, we introduce an **Adaptive Eager-Emission** framework that combines heuristic output commitment, dynamic context expansion, and overlap recovery, reducing perceived latency while preserving long-range temporal features. For efficient on-device deployment, we propose a structured compression pipeline based on encoder pruning, recovery fine-tuning, and INT8 quantization with ONNX optimization, reducing a 600M-parameter, 2.4 GB model to a 24.5M-parameter, 36 MB model while maintaining recognition performance. The streaming framework consistently outperforms static sliding-window approaches across the accuracy-latency trade-off, achieving responsiveness as low as 1.0 s at roughly 2.1% phoneme error rate (PER), or near-offline performance below 1% PER at 6.0 s average chunk latency, while the production streaming configuration maintains 99.45% average *Sifat* attribute accuracy. On a mid-range Android CPU, the compressed model runs at a real-time factor of 0.225 with a peak memory of about 600 MB.

1 Introduction

Correct Quranic recitation is governed by *Tajweed*, a system of articulation rules that determines how each letter is pronounced, elongated, nasalized, or merged with its neighbors. Mastering Tajweed traditionally requires a qualified human teacher who

listens to a learner and corrects pronunciation in real time. Teacher availability, cost, and geographic access limit how often learners can practice with immediate feedback. An automatic assessor that detects *phoneme-level* and *articulatory* errors and responds instantly could substantially broaden access to high-quality recitation practice.

Unlike generic Arabic automatic speech recognition (ASR), Quranic recitation assessment is best framed as a structured sequence-recognition problem in which orthography, elongation patterns, and letter attributes jointly define correctness. A useful assessor must recognize not only *what* was said but also *how* it was said, characterizing articulatory attributes (*Sifat*) at the phoneme level. Our underlying model addresses this with a multi-head Wav2Vec2-BERT encoder (Seamless Communication et al., 2023) trained with Connectionist Temporal Classification (CTC) (Graves et al., 2006) to predict a phoneme stream and ten parallel *Sifat* attribute streams (Abdelfattah et al., 2025).

Deploying such a model for live, on-device feedback exposes two obstacles. First, strong speech encoders are trained on *complete* utterances with full bidirectional context; used in a streaming setting, they lose the right-side context they rely on and produce systematic errors near every chunk boundary. Second, these encoders are large: our baseline occupies 2.4 GB, far beyond a practical mobile budget, and offloading inference to the cloud sacrifices privacy, compute, and offline availability. These two obstacles are the focus of this paper.

We make the following contributions:

- We introduce **Adaptive Eager-Emission**, a streaming inference framework that converts an offline full-utterance CTC model into a low-latency streaming recognizer *without re-training*. It combines dual commit heuristics, dynamic context expansion, and seam-based overlap recovery, adapting window size to

what each segment requires and guaranteeing a deterministic worst-case latency bound.

- We design a **structured compression pipeline**—four-axis encoder pruning, plain-CTC recovery fine-tuning, and INT8 ONNX export—that reduces the model from 600M parameters / 2.4 GB to 24.5M parameters / 36 MB while preserving recognition and *Sifat* accuracy.
- We provide an **empirical characterization** of the accuracy–latency trade-off and the on-device compute cost, showing effective latencies from 1.0 s responsiveness ($\sim 2.1\%$ PER) to 6.0 s ACL ($< 1\%$ PER), a real-time factor of 0.225, and ~ 600 MB peak memory on a mid-range Android CPU.

Together these results show that accurate, low-latency, privacy-preserving Quranic recitation assessment can run entirely on-device.

2 Related Work

Quranic and Arabic recitation assessment.

Most deployed Quran ASR systems target verse identification rather than pronunciation assessment. Commercial systems like Tarteel target verse identification and flag missing words, though their foundational dataset (Khan et al., 2021) lacks Tajweed annotations, meaning they do not assess Tajweed attributes at the phoneme level. More broadly, prior systems tend either to identify verses or to detect a narrow set of Tajweed phenomena, rather than characterizing articulatory attributes per phoneme, so comprehensive phoneme-level *Sifat* assessment remains rare. Our model builds on a multi-level CTC formulation that jointly predicts phonemes and *Sifat* (Abdelfattah et al., 2025); this paper focuses on making that formulation *streaming* and *on-device*.

Streaming end-to-end ASR. CTC (Graves et al., 2006) enables frame-synchronous, non-autoregressive decoding that is naturally suited to streaming, in contrast to autoregressive attention decoders (Chiu et al., 2018). Self-supervised encoders such as wav2vec 2.0 (Baevski et al., 2020), HuBERT (Hsu et al., 2021), and Conformer-based models (Gulati et al., 2020; Seamless Communication et al., 2023) achieve strong representations but are trained over complete utterances, and Flynn and Ragni (Flynn and Ragni, 2023) showed empirically

that such models benefit from up to 80 s of acoustic context—acoustic context length is therefore a first-class performance variable, not merely a computational constraint. The dominant path to streaming addresses this architecturally: encoders are retrained with look-ahead masking (Zhang et al., 2020) or sliding-window attention; Moonshine v2 (Kudlur et al., 2026) exemplifies this approach, achieving bounded time-to-first-token on edge devices by replacing global attention with an ergodic windowed encoder. An orthogonal strategy adapts the inference procedure alone, leaving the trained model unchanged. Liu et al. (Liu et al., 2020) showed that a full-sequence model can operate incrementally by committing only the stable prefix of each chunk’s output, deferring boundary-uncertain tokens until the next chunk confirms them. Our framework shares this no-retraining philosophy but inverts the commitment strategy: rather than withholding uncertain tokens, it expands the processing window to acquire the missing acoustic context—a design motivated by Quranic elongations whose temporal extent cannot be reduced without phonetic error.

Model compression for speech encoders. Structured pruning removes whole architectural units—attention heads (Michel et al., 2019), feed-forward neurons, layers, and hidden dimensions (Xia et al., 2022; Ma et al., 2023)—yielding dense models that run efficiently on commodity hardware, unlike unstructured sparsity (Sanh et al., 2020; Frankle and Carbin, 2019). Depth reduction is especially effective because deep layers are often redundant (Gromov et al., 2025; Men et al., 2025). For self-supervised speech models specifically, joint distillation and pruning (Peng et al., 2023) and structured pruning for speaker tasks (Li et al., 2025; Han et al., 2025b,a) have been explored. We compose structured pruning with low-precision INT8 inference and report a negative result on knowledge distillation (Hinton et al., 2015) under aggressive pruning.

3 Background: Multi-Head CTC Model

The assessor is built on w2v-bert-2.0 (Seamless Communication et al., 2023), a Conformer encoder (Gulati et al., 2020) that augments Transformer self-attention (Vaswani et al., 2017) with convolution, with 24 layers, hidden dimension 1024, 16 attention heads, and feed-forward dimension 4096, plus one adapter layer, totaling roughly 600M parameters. The encoder feeds **11 parallel CTC heads**: one

phoneme head and ten *Sifat* heads, each predicting an articulatory attribute (e.g., *hams/jahr* voicing, *qalqalah*, *ghunnah*, *itbaq*). Predicting phonemes and attributes from a shared encoder state avoids the latency and training cost of a sequential dependency, and CTC removes the need for frame-level alignment (Graves et al., 2006).

The frontend emits one CTC frame approximately every 40 ms (a 400-sample analysis window at 16 kHz with a 160-sample hop, followed by two stride-2 downsampling stages). Streaming paths use greedy CTC decoding to minimize latency. The reference phoneme sequence each recitation is scored against is produced by a rule-based component of the broader system (Abdelfattah et al., 2025) and is outside the scope of this paper.

4 Adaptive Eager-Emission Streaming

4.1 Problem

The encoder was trained on complete utterances with full bidirectional context. In streaming mode, phonemes near the right edge of each processed chunk lack future context and are systematically misclassified. Naively shrinking the chunk to reduce latency makes this worse (almost every token lacks context), while enlarging it restores accuracy at the cost of responsiveness. For Quranic recitation, this tension is further sharpened by obligatory vowel elongations (*Madd*) spanning two to six phonetic beats: any fixed window narrow enough for responsive feedback is too narrow to capture a full elongation, while one wide enough for *Madd* passages adds unacceptable latency to short segments. No single window size resolves this. The framework addresses it by expanding context *only when heuristics detect that more evidence is needed* and by committing output *eagerly* once it is safe.

4.2 Four-Region Adaptive Window

Each inference operates on a window with four regions (Table 1): a *left context* of committed audio that restores the encoder’s convolutional history; a dynamically sized *chunk* that is the only region allowed to emit tokens; a *right look-ahead* of real future audio used purely as encoder context; and a short *right pad* of zeros that softens convolutional boundary artifacts. The look-ahead is the single most impactful parameter because it supplies the future context that boundary phonemes require; its ablation effect is reported in Section 6.6. A

Region	Duration	Role
Left context	1.0 s	encoder history
Chunk	1.5–7.5 s	emission target
Look-ahead	1.0 s	future context only
Right pad	0.05 s	boundary smoothing

Table 1: The four-region streaming window for the production configuration. Only the chunk region emits tokens; the look-ahead provides the future context that boundary phonemes require.

decoder wrapper restricts token emission to the chunk region; frames in the look-ahead and pad regions contribute to encoder hidden states but are never decoded into output.

4.3 Dual Commit Heuristics

After decoding, two heuristics decide whether the current output can be committed or whether the window must expand:

- **H2 (trailing-run detection), evaluated first.** If the decoded sequence ends in a run of ≥ 2 identical tokens, a vowel elongation is active. Committing prematurely would truncate it, so the window expands. H2 is evaluated first, and the elongation state persists across forced commits: a *Madd* truncated at the expansion-budget boundary is detected in the immediately following chunk rather than treated as a new utterance.
- **H1 (edge-peak detection), evaluated second.** If the last token’s CTC probability peak falls within 120 ms of the chunk’s right boundary (3 CTC frames at 40 ms per frame), its acoustic evidence is truncated and the model lacks the future context for a confident decision, so the window expands.

The expansion budget constrains the total context an elongation may acquire: with increment e and maximum m expansions, a budget of $m \cdot e \geq 3.0$ s is sufficient to span a worst-case six-beat *Madd* at *mujawad* tempo. If either heuristic fires and the budget is not exhausted, the system appends e seconds of audio and re-runs inference on the wider window; otherwise it commits. This makes worst-case latency deterministic and bounded:

$$L_{\max} = b + m \cdot e + \ell, \quad (1)$$

where b is the base chunk, m the maximum expansions, e the expansion increment, and ℓ the look-ahead. For our production configuration ($b=1.5$ s, $e=1.5$ s, $m=4$, $\ell=1.0$ s) this gives $L_{\max} = 8.5$ s.

4.4 Seam Recovery

Because each new window re-processes left-context audio, the encoder naturally re-predicts phonemes it already emitted, which would prepend duplicates to every chunk. On each commit we re-decode the last ~ 400 ms of the left context, find the longest suffix of already-emitted tokens that matches a prefix of the re-decoded overlap, strip the duplicates, and append only genuinely new boundary tokens. The procedure is deliberately conservative so that it removes confirmed duplicates without discarding the continuation tokens of an active elongation.

4.5 Incremental Feedback

Waiting for a full phrase boundary before showing any feedback would frequently exceed the 2–5 s “dead zone” beyond which users perceive the system as frozen. After every commit the system immediately updates the displayed result for any word whose PER, computed by Levenshtein alignment (Levenshtein, 1966) of the hypothesis to each word’s reference phoneme span, crosses a quality threshold. A per-word lock guards against runaway error cascades: if a word’s PER exceeds 0.75, progression freezes at that word rather than propagating incorrect phonemes. A lightweight phrase classifier scores four mutually exclusive hypotheses—continuation, repetition, next verse, and noise—from the phoneme evidence and advances the session position accordingly, handling the non-linear recitation patterns common among learners.

5 On-Device Compression Pipeline

5.1 Why Pruning Is Necessary

The baseline occupies 2.4 GB in FP32. Converting to FP16 yields ~ 1.2 GB and INT8 yields ~ 600 MB—neither approaches a practical mobile budget of about 100 MB. The parameter count itself must therefore be reduced before quantization. The pipeline has three sequential stages: structured pruning, recovery fine-tuning, and INT8 ONNX export.

5.2 Four-Axis Structured Pruning

We use structured (not unstructured) pruning so the result stays dense and runs efficiently on standard mobile processors (Sanh et al., 2020; Xia et al., 2022). Pruning operates on four architectural axes:

- **Attention heads**, scored by the Frobenius norm of their query/key/value projections or by a knowledge-distillation-guided first-order Taylor importance (Michel et al., 2019; Ma et al., 2023), reducing the head count from 16 to 4.
- **Feed-forward neurons**, scored by the product of their up- and down-projection norms; because FFN blocks dominate the parameter budget, this contributes the largest share of compression, reducing the feed-forward dimension from 4096 to 1024.
- **Encoder layers**, scored by the mean input–output cosine similarity over a calibration set; near-identity layers are redundant and removed, reducing depth from 24 to 16 (Gromov et al., 2025; Men et al., 2025).
- **Hidden dimension**, scored by mean activation L_2 -norm, reduced from 1024 to 256 subject to divisibility by the retained head count (per-head dimension 64).

Because the hidden dimension is shared across hundreds of tensors, pruning it requires coordinated, constraint-checked modification of every affected projection, normalization, convolution, and adapter weight.

5.3 Recovery Fine-Tuning

Pruning disrupts learned representations, so the compressed model is retrained with the same multi-head CTC objective used originally,

$$\mathcal{L}_{\text{CTC}} = \sum_{i=1}^N \lambda_i \mathcal{L}_i, \quad (2)$$

where $\mathcal{L}_i$ and λ_i are the loss and weight of head i . We evaluated knowledge-distillation-guided recovery, matching per-layer hidden states to the unpruned teacher (Hinton et al., 2015). During *width-only* pruning this helped, consistent with prior work on speech SSL encoders (Peng et al., 2023). However, once layers and the hidden dimension were also pruned, distillation *hurt* recovery: the aggressively pruned student converges to a representation structurally different from the teacher’s, so a per-layer matching loss pulls it toward an unreachable target. Even a projection layer reconciling the dimension mismatch did not close the gap. This matches reports that depth-pruned models recover better through light fine-tuning than distillation (Gromov et al., 2025; Han et al., 2025a).

We therefore use plain CTC fine-tuning for all final configurations. The pruned model was fine-tuned for 20 epochs using a learning rate of 5×10^{-5} and a per-device batch size of 16 for both training and evaluation, without gradient checkpointing. The random seed was fixed to 42 for reproducibility.

5.4 INT8 ONNX Export and Deployment

The fine-tuned model is exported to ONNX, a framework-independent representation executed by ONNX Runtime, and quantized to INT8 during export. We find that this direct conversion preserves the fine-tuned model’s accuracy, so no additional quantization-adaptation training is required (Section 6.3). We verify numerical agreement against the FP32 reference model, then apply runtime graph optimizations and operator fusion for the target CPU. The final artifact—ONNX model plus configuration—reduces the network from 605.8M parameters / 2.4 GB to 24.5M parameters / **36 MB** (configuration h4/f1024/l16/d256, INT8).

6 Experiments

6.1 Setup

Datasets. Compression evaluation uses **QDAT Bench** (Abdelfattah et al., 2025) (159 samples, ~ 24.4 minutes) with ground-truth phoneme transcripts and per-phoneme *Sifat* annotations, and a held-out **three-Moshaf out-of-distribution** test set (Moshaf IDs 19.0, 29.0, and 30.0 from obadx/muaalem-annotated-v3). For recovery fine-tuning, the training split comprised 22 distinct Moshaf IDs from the same corpus, with 10% of the training data reserved for validation. Streaming evaluation uses a **golden streaming set** of 1,646 samples, also drawn from obadx/muaalem-annotated-v3. This set was manually curated by a domain expert from professional, clean-environment recitations of flawless quality to serve as a rigorous reference for Tajweed assessment. The code used to generate this streaming dataset is publicly available in our repository. Across these datasets, reference phoneme sequences are provided as pre-computed labels derived from the deterministic Quran Phonetic Script (QPS) (Abdelfattah et al., 2025), which translates Uthmani orthography directly into internally consistent phonetic targets. Recovery fine-tuning ran on a single NVIDIA L4 GPU for approximately 30 hours. The streaming configuration sweep (comprising 17 adaptive and 31 static configurations

evaluated across 1,646 samples) required approximately a full day of compute in total: the FP32 baseline sweep was executed on an NVIDIA RTX 3050 GPU, while the compressed INT8 ONNX sweep was executed on a 12th Gen Intel Core i5-12450H CPU.

Implementation Details. Model training and evaluation were implemented in PyTorch, utilizing the Transformers library to manage the Wav2Vec2-BERT backbone. During recovery fine-tuning, we applied dynamic acoustic augmentation to improve robustness using Audiomentations. This augmentation comprised Gaussian noise insertion (0.1–1.5% amplitude, $p = 0.4$), time stretching (0.8–1.5 $\times$ rate, $p = 0.4$), and gain transitions (−6 to +6 dB, $p = 0.4$). The complete software environment, including all package versions (e.g., ONNX Runtime, Datasets) and evaluation utilities, is fully documented in the requirements.txt file within our code repository.

Baselines. The FP32 model we compress and stream (Table 5) is the external state-of-the-art architecture of Abdelfattah et al. (2025), not an ablation of our own model; we adopt it as the starting point for our compression and streaming pipelines. We are not aware of another architecture that supports direct comparison under our task formulation. Standard Quranic ASR applications (such as Tar-teel) operate at the word or verse level, and foundational academic datasets in this domain (Khan et al., 2021) lack Tajweed annotations. Because they do not perform joint phoneme and *Sifat* prediction, they cannot be evaluated against our phoneme- and attribute-level metrics. Abdelfattah et al. (2025) is, to our knowledge, the only existing architecture that jointly predicts phonemes and all ten *Sifat* attributes, which is why we select it as our sole baseline and starting checkpoint.

Evaluation protocol. We evaluate two windowing regimes. *Static* evaluation (the baseline) sweeps fixed (w, l, r) triples—chunk width, left context, and right look-ahead—over sixteen target latencies, producing the comparison curves against which adaptive windowing is assessed. *Adaptive* evaluation sweeps seventeen named configurations across five tiers (Table 2), each parameterised by (b, e, m, ℓ) .

Both regimes report *inclusive* PER ($\overline{\text{PER}}_{\text{inc}}$)—Levenshtein distance averaged over *all* samples, including those exceeding a 15% PER thresh-

Tier	Label	Resp. (s)	WCL (s)
1	Real-time	≤ 1.0	≤ 4.0
2	Near-real-time	1.0–1.5	≤ 5.5
3	Balanced	~ 2.0	≤ 8.5
4	Accuracy-first	~ 3.0	≤ 15.0
5	Offline-like	~ 3.0	≤ 25.5

Table 2: Configuration tiers. Responsiveness = $b + \ell$; WCL = $b + m \cdot e + \ell$ (Eq. 1). All tiers maintain expansion budget $m \cdot e \geq 3.0$ s to absorb a worst-case six-beat *Madd* at mujawad tempo.

old. Reporting inclusive PER prevents aggressive low-latency configurations from gaining an unfair advantage by concentrating failures in an excluded tail. We additionally track worst-case latency ($L_{\max}$, Eq. 1), mean actual chunk latency (ACL = $\bar{L}_i + \ell$, where $\bar{L}_i$ is the mean committed chunk duration), and real-time factor (RTF). Confidence intervals use 10,000 bootstrap iterations.

6.2 Streaming Accuracy–Latency Trade-off

Figure 1 plots inclusive mean PER against responsiveness ($b + \ell$ for adaptive; chunk + right for static) for all 17 adaptive and 31 static configurations evaluated on the 1,646-sample golden streaming set. The adaptive Pareto frontier lies strictly below the static one at every responsiveness level: at 1.0 s responsiveness, the best adaptive configuration (rt_b0.5_e1.5_m2) achieves 2.10% PER versus 9.50% for the corresponding static window—a $4.5\times$ reduction. At 2.5 s the production configuration (bal_b1.5_e1.5_m4_la1.0, marked $\star$) achieves 0.95% versus 2.60% for static, a $2.7\times$ improvement. These gains arise because the adaptive framework acquires additional context precisely at the *Madd* boundaries that static windows truncate.

The configurations trace a Pareto frontier from a responsiveness of **1.0 s at $\sim 2.1\%$ PER** (WCL = 4.0 s, ACL $\approx$ 3.0 s) to near-offline **<1% PER at 6.0 s ACL**, dominating static sliding-window decoding at every operating point (Table 3). The production configuration is selected by the rule: among all configurations within 2% absolute PER_{inc} of the best-performing configuration, choose the one with minimum $L_{\max}$; this yields bal_b1.5_e1.5_m4_la1.0 (Tier 3, $L_{\max}$ = 8.5 s, ACL $\approx$ 6.0 s). This configuration also maintains 99.45% average *Sifat* attribute accuracy across all 1,646 streaming test samples, showing that low-latency streaming does not come at the cost of attribute-level recognition quality. Across this sweep, RTF stays below 0.15 for all viable con-

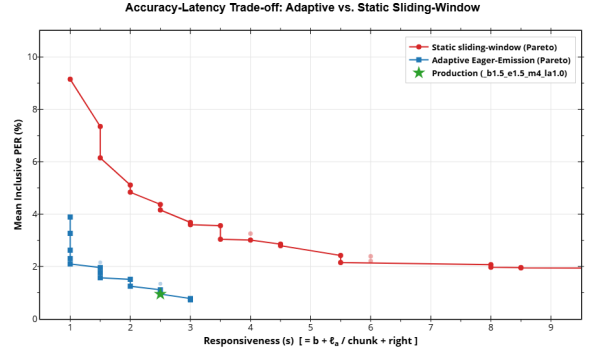


Figure 1: Accuracy–latency trade-off. Each point is one configuration. Pareto frontiers are drawn through the lowest PER at each responsiveness level. The adaptive framework (blue) dominates static sliding-window decoding (red) at every operating point. The production configuration is marked $\star$.

Approach	Latency	Accuracy
Short chunks (0.5 s)	Fixed 1 s	9.50% mean PER
Long chunks (3.0 s)	Fixed 3.5 s	2.57% mean PER
Adaptive (ours)	2.5–8.5 s	0.95% mean PER

Table 3: Comparison of windowing strategies: Static window ($left_context = 1.0$ s, $chunk = X$ s, $right_context = 0.5$ s). Adaptive windowing expands context only when the heuristics require it, dominating fixed-window decoding at every operating point. Latency range shown is responsiveness to $L_{\max}$ for the production configuration.

figurations when measured on the development server/desktop hardware used to run the large-scale configuration sweep; §6.5 reports substantially higher RTF measured natively on-device, reflecting the difference in hardware rather than a change in model or configuration. Figure 2 shows the PER distributions across representative tiers; the median is 0% on more than half the samples, confirming that errors concentrate at *Madd* boundaries.

Finally, we evaluate the 36 MB INT8 compressed model under the streaming regime to ensure the offline compression results translate to live inference. Under the production configuration (bal_b1.5_e1.5_m4_la1.0), the compressed model achieves an inclusive PER of 1.40%. This represents a slight, expected degradation of just 0.45 percentage points compared to the 0.95% PER of the uncompressed FP32 baseline. This demonstrates that the compressed model trades a marginal acoustic edge for its massive $67\times$ size reduction and remains highly robust for live on-device deployment.

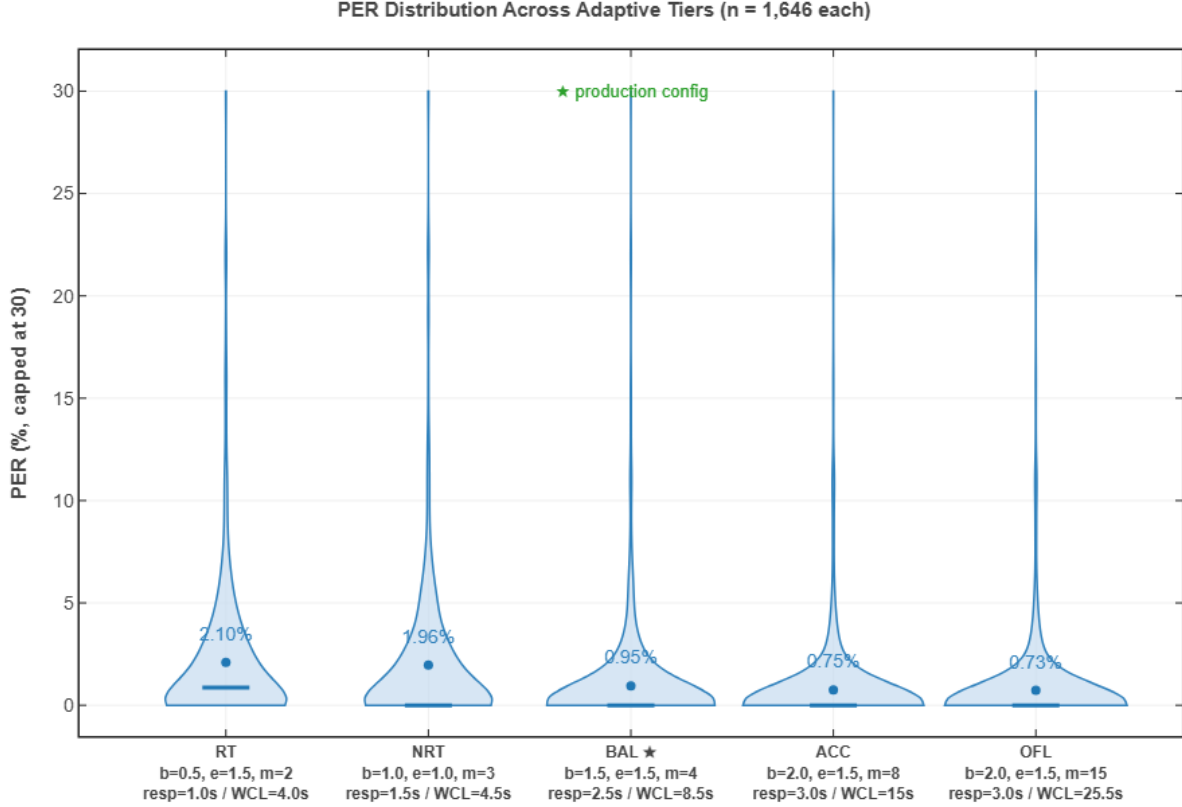


Figure 2: PER distributions for one representative adaptive configuration per tier ($n = 1,646$ samples each, violin body capped at 30%). Mean PER (shown above each violin) falls monotonically from Tier 1 (RT) to Tier 5 (OFL). The heavy median line at 0% visible in Tiers 3–5 confirms that errors concentrate in the long tail of difficult Madd passages rather than affecting typical segments. The production configuration (BAL $\star$) sits in Tier 3.

6.3 Compression, Stage by Stage

Table 5 reports QDAT Bench across the pipeline, evaluated in offline, full-utterance mode rather than under the streaming configuration of Section 6.2, so that accuracy figures isolate the effect of compression alone. 6 percentage points absolute.

The external Abdelfattah et al. (2025) FP32 model, adopted as our baseline (see Baselines paragraph, §6.1), attains 2.43% PER at 2.4 GB. After four-axis pruning and recovery fine-tuning, the 98 MB FP32 model reaches 7.27% PER with Tajweed F1 0.736 and accuracy 82.6%; INT8 ONNX export then produces the final 36 MB model at **8.05% PER** with Tajweed F1 0.700 and accuracy 78.8%—a $67\times$ size reduction. The INT8 export closely tracks the fine-tuned FP32 model’s accuracy without any quantization-adaptation training. On the cleaner out-of-distribution Moshaf set the compressed INT8 model holds **0.80% PER** (versus 0.16% for the FP32 model), indicating that the QDAT Bench gap largely reflects that benchmark’s

noisier single-verse clips rather than a true loss of capability. The pruned-before-fine-tuning model collapses (PER = 100%) and is fully restored by recovery training, underscoring the necessity of Section 5.3.

Table 4 breaks this down by phoneme recognition and each individual *Sifat* attribute on the out-of-distribution Moshaf set: every one of the ten attribute heads retains at least 98.68% accuracy after compression (worst case: Tafkheem/Taqeeq at 1.32% error), and the average error across phoneme recognition and all ten attributes rises only from 0.16% (FP32) to 0.8% (compressed INT8). This confirms that the headline PER increase observed on QDAT Bench (Table 5) is concentrated in that benchmark’s noisier single-verse clips, rather than in the *Sifat* attribute predictions that the recitation-assessment task depends on.

6.4 Robustness to Acoustic Degradation

The offline comparisons above use clean recordings. To test whether the FP32–INT8 gap stays

Metric	FP32 Err.	INT8 Err.
Phoneme Error Rate (PER)	0.54%	1.25%
Hams / Jahr	0.12%	0.70%
Shidda / Rakhawa	0.17%	1.12%
Tafkheem / Taqeeq	0.17%	1.32%
Itbaq	0.09%	0.56%
Safeer	0.13%	0.59%
Qalqla	0.09%	0.72%
Tikraar	0.09%	0.77%
Tafashie	0.16%	0.48%
Istitala	0.08%	0.56%
Ghonna	0.13%	0.78%
Average	0.16%	0.8%

Table 4: Phoneme and per-*Sifat*-attribute error rates on the out-of-distribution Moshaf set, offline mode, before (Abdelfattah et al. (2025) FP32 baseline) and after compression (INT8 ONNX). Every attribute retains at least 98.68% accuracy post-compression; the average error across all eleven metrics rises from 0.16% to 0.8%.

Stage	Size	PER
Abdelfattah et al. (2025) (FP32)	2,400 MB	2.43%
4-axis pruned + FT (FP32)	98 MB	7.27%
INT8 ONNX export	36 MB	8.05%

Table 5: Compression stages on QDAT Bench, starting from the external Abdelfattah et al. (2025) FP32 baseline. INT8 ONNX export closely preserves the fine-tuned model’s accuracy; pruning plus recovery drives the size reduction.

near its clean-condition value under realistic acoustic conditions, we evaluated both models on a 500-audio subset of the out-of-distribution Moshaf set (evaluating the full three-Moshaf set under every degradation condition was not computationally feasible) after applying controlled degradation: Speech-Shaped Noise (SSN), babble noise, and reverberation, at matched SNRs. Because this is a smaller subset than the full Moshaf set used in Table 4, its clean-condition PER (0.08% FP32, 0.86% INT8) differs slightly from the full-set average reported there (0.16%, 0.80%); both are consistent with a sub-1-point FP32–INT8 gap on clean audio.

The FP32–INT8 gap does not stay near its clean-condition value of +0.78 points: it widens to **+4.31** points under moderate reverberation (RT60 = 0.6 s) and to **+6.17** points under babble noise at 10 dB SNR—both meeting or exceeding the 5.62-point gap observed on QDAT Bench—and reaches +15.18 points under severe babble noise at 0 dB. We report this as a finding rather than a caveat: INT8 quantization measurably degrades the model’s robustness to acoustic noise, and this degradation activates under exactly the conditions—background noise and room reverberation—that a

Condition	FP32	INT8	Gap
clean	0.08%	0.86%	+0.78
ssn@20dB	0.11%	1.10%	+0.99
ssn@10dB	0.23%	2.93%	+2.70
ssn@0dB	3.76%	16.26%	+12.50
babble@20dB	0.10%	1.74%	+1.64
babble@10dB	0.69%	6.86%	+6.17
babble@0dB	13.10%	28.28%	+15.18
reverb (RT60=0.6s)	0.72%	5.03%	+4.31
reverb 0.6s + ssn@10dB	2.36%	10.88%	+8.52

Table 6: PER (%) under controlled acoustic degradation, 500-audio Moshaf subset. Gap is INT8 minus FP32, in percentage points.

phone-based tutor is likely to encounter outside a controlled recording environment. Structured pruning, recovery fine-tuning, and INT8 export preserve accuracy on clean audio, but the compressed model should not be assumed to inherit the FP32 model’s noise robustness, and deployments in acoustically uncontrolled settings should budget for substantially higher error rates than the clean-audio numbers in Table 4 suggest.

6.5 On-Device Profiling

On a Samsung Galaxy S22 (Exynos 2200) CPU the compressed model runs fully offline at **RTF 0.225** with a peak memory of about **600 MB**, comfortably faster than real time for live feedback. This figure is measured natively on mobile ARM hardware; the lower RTF (<0.15) reported in §6.2 was measured on the development server/desktop hardware used for the large-scale configuration sweep, so the two numbers reflect different evaluation hardware rather than a different model or configuration.

6.6 Analysis

Ablating the right look-ahead removes the 3–5× PER benefit of future context, confirming it as the single most impactful parameter for resolving boundary truncations. Furthermore, disabling seam recovery immediately reintroduces duplicate phonemes at every chunk boundary, validating its necessity for the eager-emission framework.

Expansion budget and elongation coverage.

The compression pipeline materially alters the model’s adaptive chunking behavior, reducing the firing rates of the edge-peak (H1) and trailing-run (H2) heuristics by 38.2% and 24.1%, respectively. To verify whether the $m \cdot e \geq 3.0$ s expansion budget still reliably covers a worst-case six-beat *Madd* despite these reduced triggers, we evaluated 1,785 reference 6-beat elongations across the dataset.

Counterintuitively, the compressed model’s reduced sensitivity actively prevents it from colliding with the expansion cap. The highly sensitive uncompressed model frequently triggers these heuristics in sequence, exhausting the budget and forcing a hard truncation before the *Madd* finishes (a 4.9% H2 cap fault rate). Because the compressed model triggers H1 and H2 substantially less, it rarely exhausts the overall budget, allowing the decoder to close chunks normally and hitting the H2 cap in only 1.0% of cases. When isolating the causes of truncated 6-beat elongations on the compressed model, acoustic under-prediction (37.6%) vastly outweighs streaming budget truncations (1.0%). This confirms that the 3.0 s budget successfully guarantees the necessary physical window, and the primary source of error is an acoustic artifact of the compressed decoder rather than a failure of the latency bounds.

7 Conclusion

We showed that accurate, low-latency, privacy-preserving Quranic recitation assessment can run entirely on a consumer phone. A retraining-free streaming framework, Adaptive Eager-Emission, turns an offline multi-head CTC model into a responsive coach with a deterministic latency bound, and a composable compression pipeline shrinks the model $67\times$ to 36 MB while preserving phoneme and *Sifat* accuracy. Figure 1 shows that the adaptive Pareto frontier uniformly dominates static sliding-window decoding: at 1.0 s responsiveness, adaptive achieves 2.10% PER versus 9.50% for the best static window at the same latency. Both techniques are general to CTC speech encoders. Future work includes additional *riwayat*, broader Tajweed coverage, and on-device personalization. Sensitivity analysis of the H1 and H2 threshold parameters, and validation of the H2 heuristic on *Ghunnah* nasalization as well as *Madd* elongation, remain open directions.

Limitations

Acoustic Robustness and Demographic Mismatch. The compressed model’s robustness to acoustic degradation is a genuine limitation. Controlled experiments (§6.4) show that INT8 quantization widens the FP32–INT8 PER gap from +0.78 points (clean audio) to +6.17 points (10 dB SNR babble noise), approaching the 5.62-point gap observed on QDAT Bench. Therefore, the com-

pressed model will likely underperform in noisy, real-world settings. Furthermore, QDAT Bench introduces a demographic mismatch: its recordings predominantly feature female voices, whereas our training data consists exclusively of male speakers. The impact of this demographic gap has not yet been fully disentangled from the aforementioned acoustic effects.

Streaming Assumptions and Heuristics. Because our streaming framework was evaluated exclusively against the golden streaming set’s professional, flawless recitations, we assume idealized input. We have not formally tested the adaptive window’s behavior on actual learner-grade audio, which typically contains mispronunciations, false starts, or extended pauses. Additionally, our H2 heuristic (trailing-run detection) was explicitly designed and validated for *Madd* elongations. We currently assume, but have not empirically verified, that it engages equally well for *Ghunnah* nasalization, which shares a multi-beat duration but differs acoustically. Threshold sensitivity optimization for these heuristics is left for future work.

Evaluation Scope and Operational Constraints. Systematically, the on-device real-time factor (RTF of 0.225) was measured in a controlled application environment, without accounting for concurrent background device loads or thermal throttling typical of extended mobile use. Operationally, while average chunk latency remains low, the balanced-tier configuration incurs a worst-case latency ($L_{\max}$) of 8.5 s, and accuracy-first configurations may reach 15.0 s, which could disrupt the user experience of instantaneous feedback during heavily elongated passages. Model-wise, while overall *Sifat* accuracy remains above 99%, certain attributes (e.g., *tafkheem* and *ghunnah*) exhibit comparatively higher error rates post-compression. Finally, all results are scoped strictly to a single recitation style (Hafs); generalization to other *qira’at* remains untested due to a lack of appropriately annotated datasets.

Data and Code Availability

To facilitate reproducibility and build upon our streaming framework, all original code, compression scripts, compressed model weights, and the generation pipeline for the golden streaming set are publicly available at <https://github.com/Abdullah-Haytham/Streaming-and-Compression-Paper>. We

release our code and compressed models under the MIT license.

Our system builds upon several existing artifacts, and our use of these assets is consistent with their intended research purposes. The foundational Wav2Vec2-BERT encoder (SeamlessM4T), the baseline FP32 model checkpoint, and the obadx/muaalem-annotated-v3 dataset from [Abdelfattah et al. \(2025\)](#) were all accessed and utilized under the MIT license. Finally, the QDAT Bench evaluation set ([Abdelfattah et al., 2025](#)) is derived from the original QDAT dataset ([Osman et al., 2021](#)), which is publicly available for research purposes without an explicitly specified license.

Ethics Statement

Automated judgment of Quranic recitation is religiously sensitive. This system is designed strictly as a self-practice aid to support, not replace, qualified human teachers. It should not be used for formal assessment, certification, or gatekeeping decisions. Furthermore, as our training data consists exclusively of adult male voices reciting in the Hafs style, deploying this model for female learners, children, or practitioners of other *qira'at* carries uncharacterized performance risks; any real-world application must address these demographic biases to ensure equitable access. Additionally, running entirely on-device, the system keeps user audio local and does not transmit recordings, protecting the privacy of learners.

Use of AI Assistants

AI-assisted tools were used to improve the clarity, grammar, and readability of the manuscript, as well as to assist in code generation. All research ideas, methodology, experiments, final software implementation, analysis, results, and conclusions were directed, verified, and thoroughly reviewed by the authors.

References

- Abdullah Abdelfattah, Mahmoud I. Khalil, and Hazem Abbas. 2025. [Automatic pronunciation error detection and correction of the Holy Quran’s learners using deep learning](#). *arXiv preprint arXiv:2509.00094*.
- Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. 2020. [wav2vec 2.0: A framework for self-supervised learning of speech representations](#). In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 12449–12460.
- Chung-Cheng Chiu, Tara N. Sainath, Yonghui Wu, Rohit Prabhavalkar, Patrick Nguyen, Zhifeng Chen, Anjali Kannan, Ron J. Weiss, Kanishka Rao, Ekaterina Gonnina, Navdeep Jaitly, Bo Li, Jan Chorowski, and Michiel Bacchiani. 2018. [State-of-the-art speech recognition with sequence-to-sequence models](#). In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4774–4778, Calgary, Canada.
- Rory Flynn and Anton Ragni. 2023. [How much context does my attention-based ASR system need?](#) *Preprint*, arXiv:2310.15672.
- Jonathan Frankle and Michael Carbin. 2019. [The lottery ticket hypothesis: Finding sparse, trainable neural networks](#). In *International Conference on Learning Representations (ICLR)*, New Orleans, Louisiana, USA.
- Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. 2006. [Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks](#). In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, Pittsburgh, Pennsylvania, USA.
- Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Gloriosi, and Daniel A. Roberts. 2025. [The unreasonable ineffectiveness of the deeper layers](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*, Singapore.
- Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. 2020. [Conformer: Convolution-augmented transformer for speech recognition](#). In *Proceedings of Interspeech*, pages 5036–5040, Shanghai, China.
- Jiangyu Han, Federico Landini, Johan Rohdin, Anna Silnova, Mireia Diez, Jan Černocký, and Lukáš Burget. 2025a. [Fine-tune before structured pruning: Towards compact and accurate self-supervised models for speaker diarization](#). In *Proceedings of Interspeech*, pages 1583–1587, Rotterdam, Netherlands.
- Jiangyu Han, Petr Pálka, Marc Delcroix, Federico Landini, Johan Rohdin, Jan Černocký, and Lukáš Burget. 2025b. [Efficient and generalizable speaker diarization via structured pruning of self-supervised models](#). *arXiv preprint arXiv:2506.18623*.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. [Distilling the knowledge in a neural network](#). *arXiv preprint arXiv:1503.02531*.
- Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. 2021. [HuBERT: Self-supervised speech representation learning by masked prediction of hidden units](#). *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460.

- Hamzah I. Khan, Abubakar Abid, Mohamed Medhat Moussa, and Anas Abou-Allaban. 2021. [The Tarteel dataset: Crowd-sourced and labeled quranic recitation](#). In *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*.
- Manjunath Kudlur, Evan King, Jiulong Wang, and Pete Warden. 2026. [Moonshine v2: Ergodic streaming encoder ASR for latency-critical speech applications](#). Preprint, arXiv:2602.12241.
- Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.
- Ze Li, Ming Cheng, and Ming Li. 2025. [Enhancing speaker verification with w2v-BERT 2.0 and knowledge distillation guided structured pruning](#). *arXiv preprint arXiv:2510.04213*.
- Danni Liu, Gerasimos Spanakis, and Jan Niehues. 2020. [Low-latency sequence-to-sequence speech recognition and translation by partial hypothesis selection](#). In *Interspeech*.
- Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. [Llm-pruner: On the structural pruning of large language models](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 21702–21720.
- Xin Men, Mingyu Xu, Qingyu Zhang, Qianhao Yuan, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2025. [ShortGPT: Layers in large language models are more redundant than you expect](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20192–20204, Vienna, Austria.
- Paul Michel, Omer Levy, and Graham Neubig. 2019. [Are sixteen heads really better than one?](#) In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 14014–14024.
- Hanaa Mohammed Osman, Ban Sharief Mustafa, and Yusra Faisal. 2021. [Qdat: a data set for reciting the quran](#). *International Journal on Islamic Applications in Computer Science And Technology*, 9(1):1–9.
- Yifan Peng, Yui Sudo, Shakeel Muhammad, and Shinji Watanabe. 2023. [DPHuBERT: Joint distillation and pruning of self-supervised speech models](#). In *Proceedings of Interspeech*, pages 62–66, Dublin, Ireland.
- Victor Sanh, Thomas Wolf, and Alexander M. Rush. 2020. [Movement pruning: Adaptive sparsity by fine-tuning](#). In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 20378–20389.
- Seamless Communication, Loïc Barrault, Yu-An Chung, and 1 others. 2023. [SeamlessM4T: Massively multilingual and multimodal machine translation](#). *arXiv preprint arXiv:2308.11596*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008.
- Mengzhou Xia, Zexuan Zhong, and Danqi Chen. 2022. [Structured pruning learns compact and accurate models](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1513–1528, Dublin, Ireland.
- Binbin Zhang, Di Wu, Zhuoyuan Yao, Xiong Wang, Fan Yu, Chao Yang, Liyong Guo, Yaguang Hu, Lei Xie, and Xin Lei. 2020. [Unified streaming and non-streaming two-pass end-to-end model for speech recognition](#). *arXiv preprint arXiv:2012.05481*.